

Developer Guide — Vermont Health Platform (HTR)

Audience: Developers setting up, extending, or deploying the platform. **Version:** 4.2.0

Table of Contents

- 1. [Prerequisites](#)
- 2. [Local Setup](#)
- 3. [Environment Variables](#)
- 4. [Running the Platform](#)
- 5. [Project Scripts](#)
- 6. [API Reference](#)
- 7. [Adding New Content Pages](#)
- 8. [Adding New Pillar Subcategories](#)
- 9. [Extending the AI Pipeline](#)
- 10. [Database Migrations](#)
- 11. [Deployment](#)
- 12. [Troubleshooting](#)

1. Prerequisites

Tool	Version	Purpose
Node.js	18.x or 20.x	Frontend runtime
npm	9.x+	Package management
Python	3.10+	Backend runtime
pip	Latest	Python packages
Git	Any	Version control

Required accounts / services (for full local functionality):

- Supabase project (free tier works for development)
- Sanity project (free tier works)
- Groq API key (free tier available)
- OpenAI API key (needed for embeddings and TTS)
- Anthropic API key (optional — only needed for advisory tier AI)
- Stripe account (optional — only needed for payment flows)

2. Local Setup

Step 1 — Clone the repository

```
git clone <repository-url>
cd Vermont-Health-Platform
```

Step 2 — Frontend setup

```
cd frontend
npm install
```

Create your environment file:

```
cp .env.production.example .env.local
# Then edit .env.local with your actual keys (see Section 3)
```

Step 3 — Backend setup

```
cd backend
python3 -m venv venv
source venv/bin/activate          # macOS/Linux
# or: venv\Scripts\activate       # Windows
pip install -r requirements.txt
```

Create your backend environment file:

```
cp .env.example .env
# Then edit .env with your actual keys (see Section 3)
```

Step 4 — Supabase setup

- 1. Create a new project at [supabase.com](#)
- 2. Enable the **pgvector** extension: Dashboard → Database → Extensions → search "vector" → enable
- 3. Run the schema SQL (see [Database Migrations](#))
- 4. Copy your project URL, anon key, service role key, and JWT secret from Project Settings → API

Step 5 — Sanity setup

- 1. Create a project at [sanity.io](#) or use the existing project ID **fxz10xl7**
- 2. Copy your project ID, dataset name, and API token from Manage → API → Tokens
- 3. Set **NEXT_PUBLIC_SANITY_PROJECT_ID**, **NEXT_PUBLIC_SANITY_DATASET**, and **SANITY_API_TOKEN**

Step 6 — Verify setup

```
# Terminal 1 – backend
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000

# Terminal 2 – frontend
cd frontend
npm run dev
```

Open <http://localhost:3000>.

3. Environment Variables

Frontend — **frontend/.env.local**

Required

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...      # Safe for browser
SUPABASE_SERVICE_ROLE_KEY=eyJ...         # Server-side only – never
```

```
expose to browser

# Sanity CMS
NEXT_PUBLIC_SANITY_PROJECT_ID=fxz10xl7
NEXT_PUBLIC_SANITY_DATASET=production
SANITY_API_TOKEN=sk... # Server-side only

# Python backend
PYTHON_BACKEND_URL=http://localhost:8000 # Dev
# PYTHON_BACKEND_URL=https://your-app.railway.app # Production

# Ingest secret (must match backend INGEST_SECRET)
INGEST_SECRET=your-shared-secret-here
```

Optional

```
# Stripe payments
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test...
STRIPE_SECRET_KEY=sk_test...
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_PRICE_SUBSCRIBER_MONTHLY=price...
STRIPE_PRICE_SUBSCRIBER_ANNUAL=price...
STRIPE_PRICE_PROFESSIONAL_MONTHLY=price...
STRIPE_PRICE_PROFESSIONAL_ANNUAL=price...

# Sanity webhook verification
SANITY_WEBHOOK_SECRET=your-webhook-secret

# Sentry error monitoring
NEXT_PUBLIC_SENTRY_DSN=https://...@sentry.io/...
SENTRY_AUTH_TOKEN=... # For source map upload at build time
SENTRY_ORG=your-org
SENTRY_PROJECT=your-project

# Transactional email (Loops.so)
LOOPS_API_KEY=...
LOOPS_WELCOME_TEMPLATE_ID=...
LOOPS_DIGEST_TEMPLATE_ID=...
```

Backend — [backend/.env](#)

Required

```
# AI / LLM
GROQ_API_KEY=gsk...
OPENAI_API_KEY=sk...

# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=eyJ...
SUPABASE_JWT_SECRET=your-jwt-secret # From: Project Settings → API → JWT Secret
SUPABASE_DB_URL=postgresql://postgres:password@db.your-project.supabase.co:5432/postgres

# Sanity
SANITY_PROJECT_ID=fxz10xl7
SANITY_DATASET=production
SANITY_API_TOKEN=sk...
SANITY_API_VERSION=2023-10-01
```

```
# Security
INGEST_SECRET=your-shared-secret-here          # Must match frontend
INGEST_SECRET

# CORS
FRONTEND_URL=http://localhost:3000              # Dev
# FRONTEND_URL=https://your-domain.com          # Production
```

Optional

```
# Advisory tier AI (Claude Sonnet 4.6)
ANTHROPIC_API_KEY=sk-ant-...

# Error monitoring
SENTRY_DSN=https://...@sentry.io/...
ENVIRONMENT=development                        # or: production

# Railway deployment
# PORT is auto-set by Railway – do NOT set manually
```

Environment Variable Reference Table

Variable	Frontend	Backend	Required	Notes
NEXT_PUBLIC_SUPABASE_URL	✓	—	Yes	Safe for browser
NEXT_PUBLIC_SUPABASE_ANON_KEY	✓	—	Yes	Safe for browser
SUPABASE_SERVICE_ROLE_KEY	✓	✓	Yes	Server-side only
SUPABASE_URL	—	✓	Yes	
SUPABASE_JWT_SECRET	—	✓	Yes	For JWT validation
SUPABASE_DB_URL	—	✓	Yes	Direct DB connection for pgvector
NEXT_PUBLIC_SANITY_PROJECT_ID	✓	—	Yes	
NEXT_PUBLIC_SANITY_DATASET	✓	—	Yes	Usually "production"
SANITY_API_TOKEN	✓	✓	Yes	Server-side in frontend
SANITY_PROJECT_ID	—	✓	Yes	Same as frontend
GROQ_API_KEY	—	✓	Yes	
OPENAI_API_KEY	—	✓	Yes	For embeddings + TTS
ANTHROPIC_API_KEY	—	✓	No	Advisory tier only
PYTHON_BACKEND_URL	✓	—	Yes	URL of FastAPI server
INGEST_SECRET	✓	✓	Yes	Must match in both
STRIPE_SECRET_KEY	✓	—	No	For payment flows
STRIPE_WEBHOOK_SECRET	✓	—	No	For webhook verification
SENTRY_DSN / NEXT_PUBLIC_SENTRY_DSN	✓	✓	No	Error monitoring

4. Running the Platform

Development

Both services must run simultaneously for full functionality.

```
# Terminal 1 – Python backend
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000

# Terminal 2 – Next.js frontend
cd frontend
npm run dev
```

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:8000>
- Backend health: <http://localhost:8000/health>
- Backend API docs (Swagger): <http://localhost:8000/docs>

Without the Backend

The frontend functions without the backend for all content browsing, pillar pages, academy content, and navigation. The following features require the backend:

- AI Analyst chat
- Personalized Learning curriculum generation
- TTS audio playback
- Follow-up question suggestions

Sanity Studio

The Sanity Studio (CMS admin) is accessible at [/studio](#) in the browser. This route is auto-excluded from the sidebar and navigation.

5. Project Scripts

Frontend ([frontend/](#))

```
npm run dev           # Start Next.js dev server (Turbopack)
npm run build          # Production build
npm run start          # Start production server
npm run lint           # ESLint
```

Backend ([backend/](#))

```
# Development
uvicorn main:app --reload --port 8000

# Production (Railway uses Procfile)
uvicorn main:app --host 0.0.0.0 --port $PORT

# Manually trigger index rebuild (requires backend running)
curl -X POST http://localhost:8000/api/ingest \
  -H "Authorization: Bearer your-ingest-secret"

# Check index rebuild status
curl http://localhost:8000/api/ingest/status \
  -H "Authorization: Bearer your-ingest-secret"

# Check health
curl http://localhost:8000/health
```

6. API Reference

Next.js API Routes (Frontend — /api/*)

All routes are in `frontend/app/api/*/route.ts`.

POST /api/chat

Proxies to Python backend. Requires valid JWT in `Authorization` header.

```
// Request
{
  message: string,           // max 2000 chars
  history: Array<{
    role: "user" | "ai",
    text: string             // max 4000 chars
  }>,                       // max 100 items
  temperature?: number,     // 0.0–1.0, default 0.7
  systemPrompt?: string     // max 800 chars
}

// Response: text/plain streaming
```

POST /api/personalized-learning

Proxies to Python backend. Requires valid JWT.

```
// Request
{
  role: string,
  topics: string[],
  difficulty: "foundational" | "intermediate" | "advanced",
  weekly_hours: string,
  goals: string,
  format_preferences?: string[]
}

// Response: JSON curriculum object
{
  weeks: Array<{
    week: number,
    theme: string,
    focus_areas: string[],
    items: Array<{
      id: string,
      type: "reading" | "case_study" | "reflection" | "knowledge_check",
      title: string,
      description: string,
      content: string,
      platform_link: string,
      relevance_bridge?: string,
      estimated_minutes: number,
      key_concepts: string[],
      reflection_question?: string,
      questions?: Array<{
        question: string,
        options: string[],
        correct: number,
        explanation: string
      }>
    }>
  }>
```

```
    }>
  }>
}
```

GET /api/ticker

Returns scrolling headline data for the ticker strip.

```
// Response
{
  headlines: Array<{
    text: string,
    url: string
  }>
}
```

GET /api/health

System health check.

```
// Response
{
  status: "ok",
  frontend: "up",
  backend?: "up" | "down"
}
```

POST /api/stripe/checkout

Creates a Stripe checkout session.

```
// Request
{ priceId: string, successUrl: string, cancelUrl: string }

// Response
{ url: string } // redirect to Stripe
```

POST /api/stripe/portal

Creates a Stripe billing portal session.

```
// Response
{ url: string } // redirect to Stripe portal
```

POST /api/stripe/webhook

Stripe webhook handler. Requires `stripe-signature` header.

Python Backend API Routes (/api/*)

All documented in Swagger at <http://localhost:8000/docs> when running.

POST /api/chat

```
Auth: Bearer JWT
Rate: 30/min

Request body:
{
  "message": "string",
  "history": [{"role": "user|ai", "text": "string"}],
  "temperature": 0.7,
  "systemPrompt": "string"
}

Response: text/plain (streaming)
```

POST /api/suggest

```
Auth: optional
Rate: 60/min

Request body:
{ "topic": "string" }

Response:
{ "suggestions": ["string", "string", "string"] }
```

POST /api/ingest

```
Auth: Bearer INGEST_SECRET
Rate: 5/hour

Response: 202 Accepted
{ "job_id": "uuid", "status": "queued" }
```

GET /api/ingest/status

```
Auth: Bearer INGEST_SECRET

Response:
{
  "status": "queued|running|completed|failed",
  "started_at": "ISO8601",
  "completed_at": "ISO8601",
  "error_message": "string|null"
}
```

GET /api/v1/states

```
Auth: X-HTR-API-Key header

Response:
{
  "data": [RhtStateProfile, ...],
  "count": 50
}
```


GET /api/v1/states/{state_id}

Auth: X-HTR-API-Key header

Response:

```
{
  "data": {
    "state_id": "vermont",
    "state_name": "Vermont",
    "performance_score": 42,
    "cost_index": 1.18,
    "quality_score": 38,
    "access_score": 51,
    "equity_score": 44,
    "innovation_score": 39,
    "preventive_care_rate": 0.67,
    "uninsured_rate": 0.042,
    "data_year": 2024
  }
}
```

POST /api/personalized-learning/generate

Auth: Bearer JWT

Request body:

```
{
  "role": "Hospital Administrator",
  "topics": ["Value-Based Care", "SDOH Integration"],
  "difficulty": "intermediate",
  "weekly_hours": "3-5",
  "goals": "Understand AHEAD model implications",
  "format_preferences": ["readings", "case_studies"]
}
```

Response: JSON curriculum (see frontend /api/personalized-learning schema above)

POST /api/personalized-learning/tts

Auth: Bearer JWT

Request body:

```
{
  "text": "string to speak",
  "voice": "alloy|echo|fable|onyx|nova|shimmer"
}
```

Response: audio/mpeg (MP3 stream)

7. Adding New Content Pages

New Pillar Subcategory Page

Example: Adding **/policy/interstate-compact**s

1. Create the page file:
-

```
// frontend/app/policy/interstate-compact/page.tsx
import Link from "next/link";
import HubSubscribeCTA from "@components/HubSubscribeCTA";
import LatestHubReports from "@components/LatestHubReports";

export default function Page() {
  const topics = [
    {
      label: "Nurse Licensure Compact",
      href: "/policy/interstate-compact/nlc",
      description: "Multi-state nursing licensure framework.",
      details: ["License Portability", "State Participation", "Enforcement"],
      scope: "Analysis of the NLC and its implications for workforce mobility."
    },
    // ... more subtopics
  ];

  return (
    <div className="max-w-7xl mx-auto px-6 py-12">
      {/* Header */}
      <div className="mb-12">
        <span className="text-sm font-bold text-sky-700 uppercase tracking-wider">
          Health Policy
        </span>
        <h1 className="text-4xl font-black text-slate-900 mt-2 mb-4">
          Interstate Compacts
        </h1>
        <p className="text-xl text-slate-600 max-w-3xl">
          Multi-state agreements shaping healthcare workforce and data sharing.
        </p>
      </div>

      {/* Topic Cards */}
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
        {topics.map((item) => (
          <Link key={item.label} href={item.href}
            className="group flex flex-col p-6 bg-white rounded-2xl border
border-slate-200 shadow-sm hover:shadow-lg transition-all hover:-translate-y-1
hover:border-sky-400 hover:bg-sky-50/80">
            {/* ... card contents */}
          </Link>
        ))}
      </div>

      {/* Tools & Data Section */}
      <div className="mt-16 mb-12">
        {/* Add relevant tool crosslinks here */}
      </div>

      <LatestHubReports pillar="Policy" colorClass="text-sky-700"
        cardHoverClass="hover:border-sky-400 hover:bg-sky-50/80"
        titleHoverClass="group-hover:text-sky-700" />

      <HubSubscribeCTA pillar="Policy" bgClass="bg-sky-50"
        buttonClass="bg-sky-700 hover:bg-sky-800" />
    </div>
  );
}
```

2. **Add to the sidebar** (`HomeSidebar.tsx`) under the Policy pillar's `items` array:

```
{ href: "/policy/interstate-compact", label: "Interstate Compacts" },
```

3. **Add to Header mega-menu** (`Header.tsx`) in the `pillars` array under policy `items`:

```
{ href: "/policy/interstate-compacts", label: "Interstate Compacts" },
```

4. **Add to sitemap** (`app/sitemap.ts`):

```
{ url: `${base}/policy/interstate-compacts`, lastModified: new Date(),
  priority: 0.7 }
```

New Hub Page (Tabbed)

For a full hub page with tabs, use `HubPageTemplate`:

```
// frontend/app/my-new-hub/page.tsx
import HubPageTemplate from "@components/templates/HubPageTemplate";
import { SomeIcon } from "@heroicons/react/24/outline";
import MyTabContent from "./MyTabContent";

export default function MyNewHub() {
  const tabs = [
    {
      id: "overview",
      label: "Overview",
      icon: <SomeIcon className="w-5 h-5" />,
      content: <MyTabContent />
    },
    // more tabs...
  ];

  return (
    <HubPageTemplate
      badgeLabel="My Hub"
      title="Hub Title"
      subtitle="Hub description for healthcare leaders."
      tabs={tabs}
    />
  );
}
```

8. Adding New Pillar Subcategories

To add a new subcategory that appears in both the header mega-menu and the left sidebar, update these three locations:

1. `frontend/components/Header.tsx`

Find the relevant pillar in the `pillars` array and add to `items`:

```
const pillars = [
  {
    id: "policy",
    // ...
    items: [
      // existing items...
      { href: "/policy/my-new-topic", label: "My New Topic" }, // ← add here
    ],
  },
];
```

```
    },
    // ...
  ];
```

2. frontend/components/HomeSidebar.tsx

Find the relevant pillar in `pillarData` and add to `items`:

```
const pillarData = [
  {
    id: "policy",
    // ...
    items: [
      // existing items...
      { href: "/policy/my-new-topic", label: "My New Topic" }, // ← add here
    ],
  },
  // ...
];
```

3. Mobile menu in `Header.tsx`

The mobile menu is built from the `pillars` array automatically — no separate update needed.

9. Extending the AI Pipeline

Adding a New Agentic Tool

Tools are defined in `backend/services/tools.py`. Add a new function decorated with LlamaIndex's `FunctionTool`:

```
from llama_index.core.tools import FunctionTool

def my_new_tool(input_param: str) -> str:
    """
    Description of what this tool does – used by the agent to decide when to
    call it.

    Args:
        input_param: Description of the parameter
    """
    # Your logic here
    result = fetch_some_data(input_param)
    return f"Result: {result}"

my_new_tool_fn = FunctionTool.from_defaults(fn=my_new_tool)
```

Then register it in the agent creation in `routers/chat.py`:

```
from services.tools import query_state_metrics_fn, list_research_lab_tools_fn,
my_new_tool_fn

agent = ReActAgent.from_tools(
    tools=[query_state_metrics_fn, list_research_lab_tools_fn, my_new_tool_fn],
    llm=llm,
    max_iterations=5,
)
```

Adding New Content Types to the Index

In `backend/services/indexing.py`, add a new GROQ query in `fetch_sanity_content()`:

```
async def fetch_sanity_content():
    queries = {
        "myNewType": """*[_type == "myNewType"]{
            title,
            body,
            "pillar": pillar,
            "slug": slug.current
        }"""
    }
    # The function iterates over queries and converts each to LlamaIndex
    Documents
```

After adding, trigger a reindex:

```
curl -X POST http://localhost:8000/api/ingest \
-H "Authorization: Bearer your-ingest-secret"
```

Changing the System Prompt

The AI Analyst system prompt is in `routers/chat.py`. It instructs the model to behave as an HTR analyst:

```
SYSTEM_PROMPT = """You are the HTR AI Analyst – an expert in U.S. healthcare
transformation across policy, economics, technology, clinical innovation, and
health equity. Answer questions using only the provided context...
"""
```

Modify this string to adjust tone, persona, output format, or citation behavior.

10. Database Migrations

Initial Schema Setup

Run this SQL in your Supabase SQL editor (Dashboard → SQL Editor):

```
-- Enable pgvector
CREATE EXTENSION IF NOT EXISTS vector;

-- Profiles
CREATE TABLE profiles (
    id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
    full_name TEXT,
    avatar_url TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can view own profile" ON profiles FOR SELECT USING
(auth.uid() = id);
CREATE POLICY "Users can update own profile" ON profiles FOR UPDATE USING
(auth.uid() = id);

-- User roles
CREATE TABLE user_roles (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    role TEXT CHECK (role IN
('free','subscriber','student','professional','advisory','admin')),
    granted_at TIMESTAMPTZ DEFAULT NOW(),
    granted_by UUID
);
ALTER TABLE user_roles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can view own roles" ON user_roles FOR SELECT USING
(auth.uid() = user_id);

-- Subscriptions
CREATE TABLE subscriptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    stripe_customer_id TEXT,
    stripe_sub_id TEXT,
    plan TEXT,
    status TEXT,
    current_period_end TIMESTAMPTZ,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can view own subscription" ON subscriptions FOR SELECT
USING (auth.uid() = user_id);

-- Conversations
CREATE TABLE conversations (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    title TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE conversations ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own conversations" ON conversations
USING (auth.uid() = user_id);

-- Conversation messages
CREATE TABLE conversation_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    conversation_id UUID REFERENCES conversations(id) ON DELETE CASCADE,
    role TEXT CHECK (role IN ('user','assistant')),
    content TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE conversation_messages ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own messages" ON conversation_messages
USING (
    conversation_id IN (
        SELECT id FROM conversations WHERE user_id = auth.uid()
    )
);

-- RAG documents (vector store)
CREATE TABLE rag_documents (
    id BIGSERIAL PRIMARY KEY,
    content TEXT NOT NULL,
    metadata JSONB DEFAULT '{}',
    embedding VECTOR(1536),
    created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX rag_documents_embedding_idx ON rag_documents
USING hnsw (embedding vector_cosine_ops);
CREATE INDEX rag_documents_content_fts ON rag_documents
USING gin (to_tsvector('english', content));
```

```
-- RAG query log
CREATE TABLE rag_query_log (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id TEXT,
  query TEXT,
  role TEXT,
  model_used TEXT,
  retrieved_doc_ids TEXT[],
  retrieved_scores FLOAT[],
  response_preview TEXT,
  latency_ms INTEGER,
  was_zero_result BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- API keys
CREATE TABLE api_keys (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  key_hash TEXT UNIQUE,
  tier TEXT DEFAULT 'researcher',
  requests_today INTEGER DEFAULT 0,
  last_used_at TIMESTAMPTZ,
  revoked_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
ALTER TABLE api_keys ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can manage own API keys" ON api_keys
  USING (auth.uid() = user_id);

-- Ingest jobs
CREATE TABLE ingest_jobs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  status TEXT CHECK (status IN ('queued', 'running', 'completed', 'failed')),
  started_at TIMESTAMPTZ,
  completed_at TIMESTAMPTZ,
  error_message TEXT
);

-- State performance index
CREATE TABLE state_performance_index (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  state_name TEXT,
  state_id TEXT UNIQUE,
  performance_score FLOAT,
  cost_index FLOAT,
  quality_score FLOAT,
  access_score FLOAT,
  equity_score FLOAT,
  innovation_score FLOAT,
  preventive_care_rate FLOAT,
  uninsured_rate FLOAT,
  data_year INTEGER,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- RHT state profiles
CREATE TABLE rht_state_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  state_id TEXT UNIQUE,
  state_name TEXT,
  award_amount TEXT,
  status TEXT,
  strategic_focus TEXT,
  description TEXT,
  initiatives JSONB DEFAULT '[]',
```

```
    metrics JSONB DEFAULT '[]',
    simulation JSONB
);

-- Survey aggregate
CREATE TABLE survey_aggregate (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    edition TEXT,
    results JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Hybrid search function (required by retrieval service)
CREATE OR REPLACE FUNCTION hybrid_search_rag(
    query_embedding VECTOR(1536),
    query_text TEXT,
    match_count INT DEFAULT 20
)
RETURNS TABLE (
    id BIGINT,
    content TEXT,
    metadata JSONB,
    score FLOAT
)
LANGUAGE plpgsql AS $$
DECLARE
    dense_weight FLOAT := 0.6;
    sparse_weight FLOAT := 0.4;
BEGIN
    RETURN QUERY
    WITH dense AS (
        SELECT
            rd.id,
            rd.content,
            rd.metadata,
            1 - (rd.embedding <=> query_embedding) AS score,
            ROW_NUMBER() OVER (ORDER BY rd.embedding <=> query_embedding) AS rank
        FROM rag_documents rd
        WHERE rd.embedding IS NOT NULL
        LIMIT match_count * 2
    ),
    sparse AS (
        SELECT
            rd.id,
            rd.content,
            rd.metadata,
            ts_rank(to_tsvector('english', rd.content), plainto_tsquery('english',
query_text)) AS score,
            ROW_NUMBER() OVER (
                ORDER BY ts_rank(to_tsvector('english', rd.content),
plainto_tsquery('english', query_text)) DESC
            ) AS rank
        FROM rag_documents rd
        WHERE to_tsvector('english', rd.content) @@ plainto_tsquery('english',
query_text)
        LIMIT match_count * 2
    ),
    rrf AS (
        SELECT
            COALESCE(d.id, s.id) AS id,
            COALESCE(d.content, s.content) AS content,
            COALESCE(d.metadata, s.metadata) AS metadata,
            (
                COALESCE(dense_weight / (60.0 + d.rank), 0) +
                COALESCE(sparse_weight / (60.0 + s.rank), 0)
            ) AS score
    )
```



```
FROM dense d
FULL OUTER JOIN sparse s ON d.id = s.id
)
SELECT rrf.id, rrf.content, rrf.metadata, rrf.score
FROM rrf
ORDER BY rrf.score DESC
LIMIT match_count;
END;
$;
```

11. Deployment

Frontend — Vercel

1. Connect repository to Vercel
2. Set root directory to `frontend/`
3. Build command: `npm run build`
4. Output directory: `.next`
5. Add all frontend environment variables in Vercel dashboard → Settings → Environment Variables
6. Set `PYTHON_BACKEND_URL` to your Railway backend URL

Automatic deployments: Every push to `main` triggers a Vercel production deployment.

Backend — Railway

1. Create a new Railway project
2. Connect repository, set root directory to `backend/`
3. Railway uses `Procfile` for the start command:

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

4. Add all backend environment variables in Railway → Variables
5. `PORT` is auto-set by Railway — do not override it

Note: Railway auto-deploys on push to `main`. The backend URL will be `https://your-app.railway.app` — set this as `PYTHON_BACKEND_URL` in Vercel.

Stripe Webhooks

After deployment, configure Stripe to send webhook events to your production URL:

1. Stripe Dashboard → Developers → Webhooks → Add endpoint
2. URL: `https://your-domain.com/api/stripe/webhook`
3. Events: `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`
4. Copy the webhook signing secret → set as `STRIPE_WEBHOOK_SECRET`

Sanity Webhooks (Content Sync)

1. Sanity Manage → your project → API → Webhooks → Add
2. URL: `https://your-domain.com/api/webhooks/sanity`
3. Dataset: `production`
4. Trigger on: Create, Update, Delete
5. Secret: set a random string → set as `SANITY_WEBHOOK_SECRET` in Vercel

12. Troubleshooting

"Connection error. Please try again." in AI Analyst

Cause: Python backend is not running.

```
cd backend
source venv/bin/activate
uvicorn main:app --reload --port 8000
```

"Cannot coerce the result to a single JSON object" in Dashboard

Cause: `state_id` has duplicate rows in `rht_state_profiles` table. The `getRhtProfile()` function uses `.maybeSingle()` which handles this gracefully (returns first match). Deduplicate the table if needed:

```
DELETE FROM rht_state_profiles a
USING rht_state_profiles b
WHERE a.id < b.id AND a.state_id = b.state_id;
```

"Invalid JWT" errors from backend

Cause: `SUPABASE_JWT_SECRET` mismatch. Verify it matches exactly what's in Supabase → Project Settings → API → JWT Secret.

In dev mode, if you omit `SUPABASE_JWT_SECRET` from the backend `.env`, all requests get `role="subscriber"` automatically (dev bypass).

Index not finding relevant content

Cause: Index not built, or new content not yet ingested.

```
curl -X POST http://localhost:8000/api/ingest \
-H "Authorization: Bearer your-ingest-secret"

# Poll status
curl http://localhost:8000/api/ingest/status \
-H "Authorization: Bearer your-ingest-secret"
```

FlashRank taking long on first startup

Cause: FlashRank downloads its cross-encoder model to `/tmp/flashrank` on first use. This is a one-time ~100MB download. Subsequent startups are fast.

Supabase pgvector extension error

```
-- Run in Supabase SQL Editor
CREATE EXTENSION IF NOT EXISTS vector;
```

Sanity content not appearing

1. Verify `SANITY_API_TOKEN` has read access to the dataset
2. Verify `NEXT_PUBLIC_SANITY_PROJECT_ID` matches your actual project ID
3. Check Sanity Studio at `/studio` to confirm content exists
4. Trigger a manual reindex: `POST /api/ingest`

Dark mode not persisting

Dark mode state is managed by `ThemeProvider`. If it resets on refresh, check that the theme is being persisted to `localStorage` correctly. The provider reads from `localStorage.getItem("theme")` on mount.

Left sidebar not staying open between pages

The sidebar now persists its state across all pages (this was changed intentionally). If it's not persisting, check that `SidebarContext` is initialized with `isLeftOpen: true` as the default and that `AppShell` is not overriding it. The auto-collapse only fires for `/studio` and `/chat`.